MLOps (AIMLCZG523) - Assignment 1

Heart Disease Risk Prediction

An end-to-end MLOps pipeline: data, training, tracking, CI/CD, Docker, Kubernetes and monitoring

Submitted by	Pranay Jain
Student ID	2025AE05025
Email	2025ae05025@wilp.bits-pilani.ac.in
GitHub repository	https://github.com/PranayAJain/heart-disease-mlops
Demo video	Google Drive - MLOps_Demo_2025AE05025.mp4
Date	October 2026

1. Overview and architecture

I built a heart-disease risk classifier on the UCI Heart Disease (Cleveland) dataset and took it all the way from raw data to a monitored API running on Kubernetes. Every step is scripted and reproducible from the repository: one command downloads the data, one trains and tracks all models, one packages the winner, and the GitHub Actions pipeline re-runs the whole chain (lint, tests, training, packaging, quality gate) on every push.

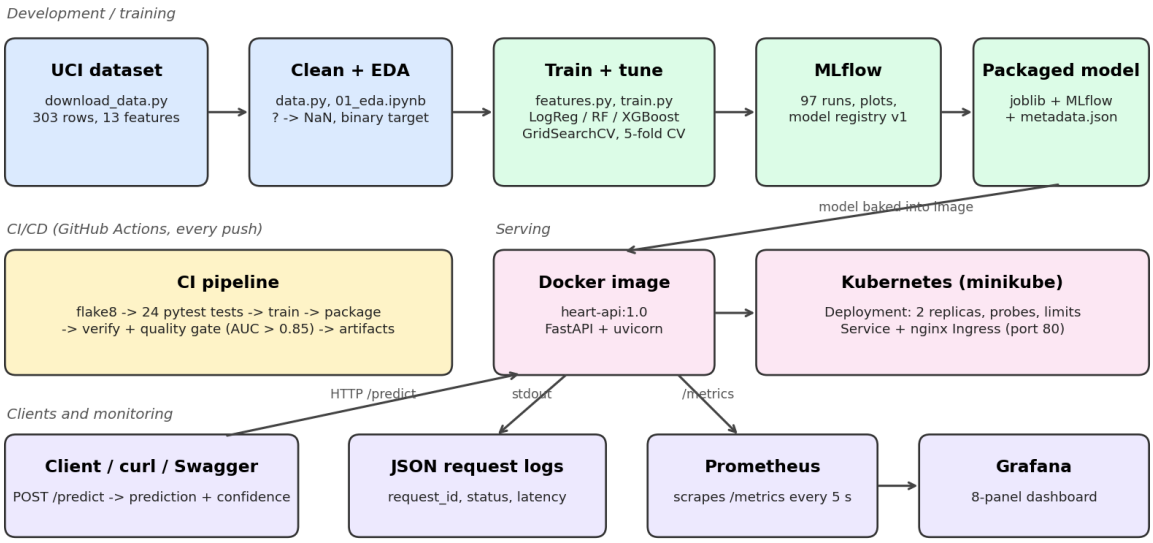


Figure 1 - System architecture

2. Data acquisition and EDA

`scripts/download_data.py` downloads the official UCI zip (dataset id 45), extracts `processed.cleveland.data` and prints the row count and an md5 fingerprint so the exact data version is traceable. The file has 303 patients, 13 clinical features and a severity label `num` (0-4).

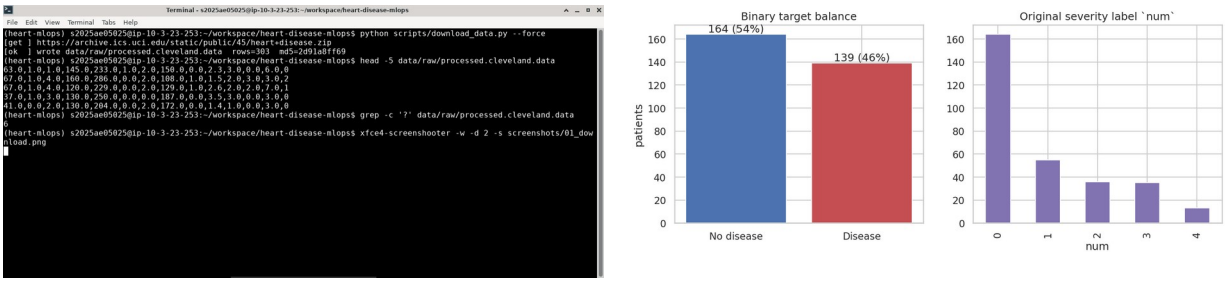


Figure 2 - Dataset download, 303 rows (left) and class balance, 164 vs 139 (right)

Cleaning decisions (src/heart/data.py)

- '?' markers become NaN. Only 6 values are missing: ca (4) and thal (2).
- Missing values are not filled during cleaning. They are imputed inside the sklearn pipeline, fitted on training folds only, so there is no leakage and the deployed API handles a missing field the same way.

- num 0-4 becomes a binary target (0 = no disease, 1 = disease). Severity levels 3 and 4 have only 35 and 13 patients, too few to predict separately.
- Integer-coded columns are cast to nullable integers, exact duplicates are dropped, and rows outside plausible physiological ranges would be dropped (none were in this dataset).

Key findings from the EDA

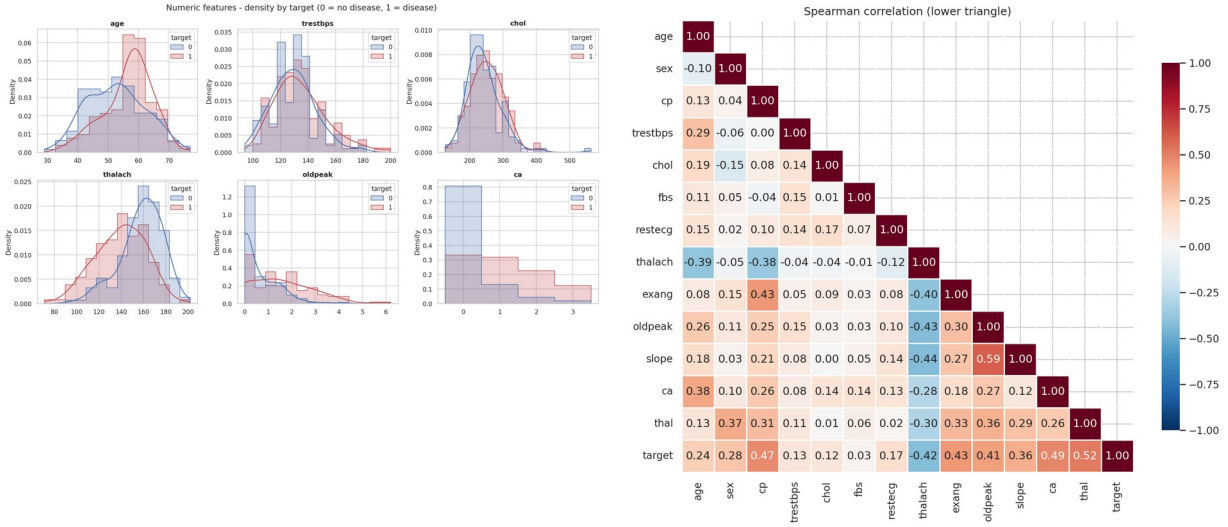


Figure 3 - Numeric feature distributions by target (left) and Spearman correlation heatmap (right)

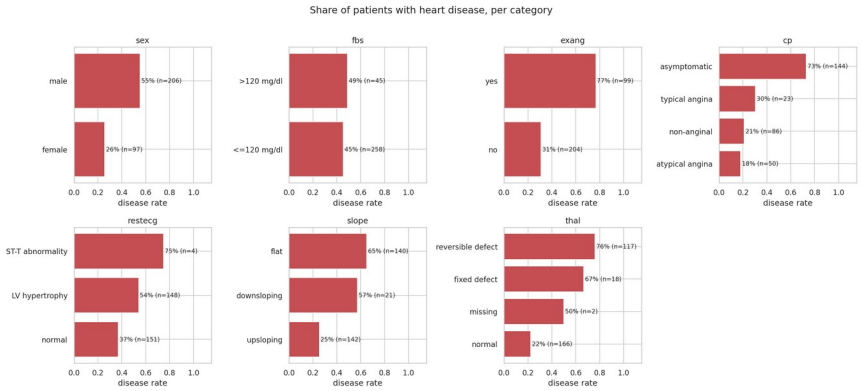


Figure 4 - Disease rate per category

- Class balance: 164 without disease (54%) vs 139 with (46%). Balanced enough that no resampling is needed; I still track recall and ROC-AUC because missing a sick patient costs more than a false alarm.
- Strongest signals: thal reversible defect (76% disease vs 22% for normal), asymptomatic chest pain (73%), exercise-induced angina (77% vs 31%), number of vessels ca, and ST depression oldpeak. Max heart rate thalach is the strongest negative signal.
- Weak signals: fasting blood sugar (49% vs 45%), cholesterol and resting blood pressure (|corr| about 0.12).
- Outliers (cholesterol > 500, oldpeak > 6) are physiologically possible and inside the valid ranges, so I kept them.
- No feature pair is correlated above 0.7, so multicollinearity is not a serious concern for logistic regression.

3. Feature engineering and model development

All preprocessing lives in one sklearn Pipeline (src/heart/features.py), so exactly the same transforms run during cross-validation, final training and inside the API:

- Numeric features: median imputation, then StandardScaler.
- Binary features: most-frequent imputation.
- Nominal features (cp, restecg, slope, thal): most-frequent imputation, then one-hot encoding with handle_unknown='ignore', so an unseen category cannot crash the service.
- Engineered feature hr_pct_max = thalach / (220 - age): the share of the age-predicted maximum heart rate the patient reached during exercise. A raw heart rate of 150 means something very different at age 30 and at age 70; this ratio puts patients on a common scale.

The 13 input columns become 23 model features. Evaluation protocol (src/heart/train.py): a stratified 80/20 split, GridSearchCV with 5-fold stratified CV optimising ROC-AUC on the training part, re-scoring of each best configuration on 5 metrics, and one final evaluation on the held-out test set. The winner is chosen by CV ROC-AUC, not by test score, to avoid overfitting to 61 test rows.

Model	Configs tried	CV ROC-AUC	CV Accuracy	CV Precision	CV Recall	Test ROC-AUC
Logistic Regression	**10**	**0.908 ± 0.019**	**0.847**	**0.879**	**0.783**	**0.960**
Random Forest	36	0.900 ± 0.023	0.818	0.839	0.764	0.953
XGBoost	48	0.889 ± 0.020	0.818	0.835	0.765	0.959

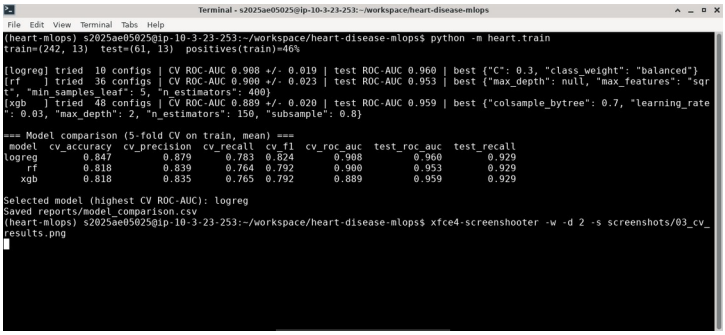


Figure 5 - Cross-validation and tuning output

Model selection

Logistic regression (C = 0.3, class_weight = 'balanced') wins on CV ROC-AUC and on every other CV metric. The gaps (0.01-0.02) are within one standard deviation, so the three models are statistically close; on 242 training rows the simplest model generalises best, and it is also the most interpretable, which matters for a clinical use case. Tuning chose strong regularisation everywhere (small C, min_samples_leaf = 5, XGBoost max_depth = 2 with learning rate 0.03), a sign that this small dataset rewards simple models. Test ROC-AUC (about 0.96) is higher than CV (about 0.91) only because of the variance of a 61-row test set.

Results and inference on the held-out test set

The selected model was evaluated once on the 61 held-out patients (33 without disease, 28 with):

	Predicted: no disease	Predicted: disease
Actual: no disease (33)	27 (true negatives)	6 (false positives)
Actual: disease (28)	2 (false negatives)	26 (true positives)

Test metric	Value	What it means here
Recall (sensitivity)	0.929	26 of 28 sick patients are flagged; only 2 are missed
Precision	0.813	about 4 in 5 positive flags are real disease
Accuracy	0.869	53 of 61 patients classified correctly
F1	0.867	balance of precision and recall
ROC-AUC	0.960	ranking quality across all thresholds

Inference: for a screening tool this error profile is the right one. class_weight = 'balanced' pushes the model towards catching disease, so it trades 6 false alarms (which lead to a follow-up test) for only 2 missed patients (which could lead to an untreated condition). If an even lower miss rate were required, the 0.5 decision threshold stored in model_metadata.json could be lowered without retraining.

What drives the predictions (standardised logistic-regression coefficients of the packaged model): the largest risk-increasing factors are the number of major vessels coloured (ca, +0.93), asymptomatic chest pain (cp = 4, +0.91), male sex (+0.84), reversible thal defect (+0.68), exercise-induced angina (+0.57) and ST depression (oldpeak, +0.39). The strongest protective factors are a normal thal result (-0.54), non-anginal or typical chest pain (-0.51 / -0.39), an upsloping ST segment (-0.35) and a higher maximum heart rate (thalach -0.23, hr_pct_max -0.15). These match the EDA findings and clinical intuition, which gives confidence the model learned real signal rather than noise.

Example predictions from the packaged model: a 62-year-old man with asymptomatic chest pain, exercise angina and a reversible defect gets p(disease) = 0.996; a 41-year-old woman with atypical angina, normal thal and a high max heart rate gets 0.03; a record with ca and thal missing still gets a valid prediction because the pipeline imputes them.

4. Experiment tracking with MLflow

src/heart/tracking.py logs one parent run per model plus one nested child run for every grid-search configuration (94 children, 97 runs in total). Each parent run records:

- Parameters: model kind, best hyperparameters, CV folds, seed, train/test sizes and an md5 of the cleaned data (data versioning).
- Metrics: CV mean and standard deviation for 5 metrics, plus held-out test metrics.
- Artifacts: confusion matrix, ROC curve, feature-importance plot and table, the full grid-search table, and the fitted pipeline with an input signature.
- Tags: author, stage and selection metric, so runs can be filtered later.

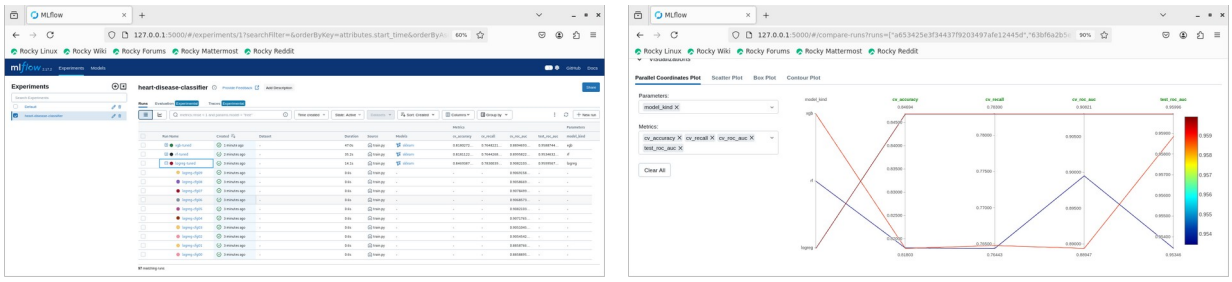


Figure 6 - MLflow runs with nested grid-search runs (left); comparing the 3 tuned models (right)

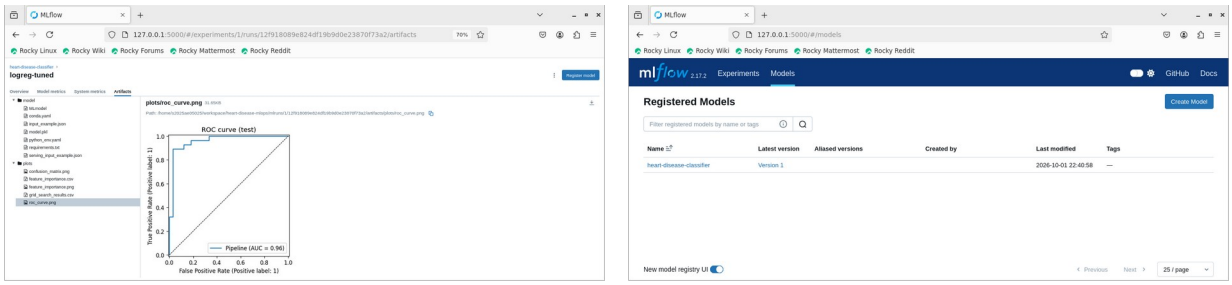


Figure 7 - Logged artifacts with ROC curve, AUC 0.96 (left); model registered in the MLflow Model Registry (right)

5. Model packaging and reproducibility

src/heart/package.py reads the comparison table, rebuilds the winning pipeline with its best parameters, re-fits it on all 303 rows and saves it in three formats:

Artifact	Purpose
models/heart_model.joblib	Full sklearn Pipeline (preprocessing + classifier); loaded by the API
models/mlflow_model/	MLflow format with MLmodel, conda.yaml, requirements.txt and input example
models/model_metadata.json	Version, parameters, CV and test metrics, feature list, data md5, library versions, decision threshold
MLflow Model Registry	Registered as heart-disease-classifier (versioned)

Inference script: python -m heart.predict samples/patient_high_risk.json prints the prediction, probability and confidence for one or more patient JSON files, using the same helper the API uses.

Because preprocessing is inside the saved pipeline, a client only ever sends the 13 raw clinical fields. scripts/verify_model.py reloads both formats in a fresh process, predicts for a high-risk, a low-risk and an incomplete patient (ca and thal missing), and confirms both formats return identical probabilities. Library versions are pinned in requirements.txt; the environment was rebuilt from scratch on a new lab machine with conda create + pip install -r requirements.txt and all tests passed unchanged.

6. CI/CD pipeline and automated testing

The test suite has 24 pytest tests, all running offline on a small synthetic fixture:

File	Tests	What is checked
test_data.py	7	Binary target mapping, duplicate and out-of-range removal, missing values kept for

File	Tests	What is checked
		imputation, dtypes, '?' parsing, clear error for a missing file
test_features.py	5	hr_pct_max formula, no NaNs after the pipeline, scaling, unseen category handled, one-hot feature names
test_model.py	5	All model specs build, probabilities in [0,1], prediction contract, packaged model flags a high-risk patient, quality gate AUC > 0.85
test_api.py	7	/health, /predict response and request id, optional fields, 422 on invalid or missing input, batch endpoint, /metrics

The GitHub Actions workflow (.github/workflows/ci.yml) runs on every push and pull request in two jobs. Job 1 lints with flake8 and runs the full test suite, uploading the JUnit report even on failure. Job 2 only runs if job 1 passes: it downloads the data (falling back to the committed cleaned CSV if UCI is unreachable), trains and tunes all models with MLflow tracking, packages the model, runs the reload check and quality gate, and uploads the model, training log and MLflow runs as artifacts. Any non-zero exit code turns the run red.

To prove the pipeline really fails on bad code, I pushed a deliberate bug to a separate branch (target mapping num > 1 instead of num > 0). The unit test test_severity_levels_map_to_disease caught it, the lint-and-test job went red in 1 min 25 s, and the training job was skipped, so a broken model was never trained or packaged. The branch was then deleted and main stayed green.

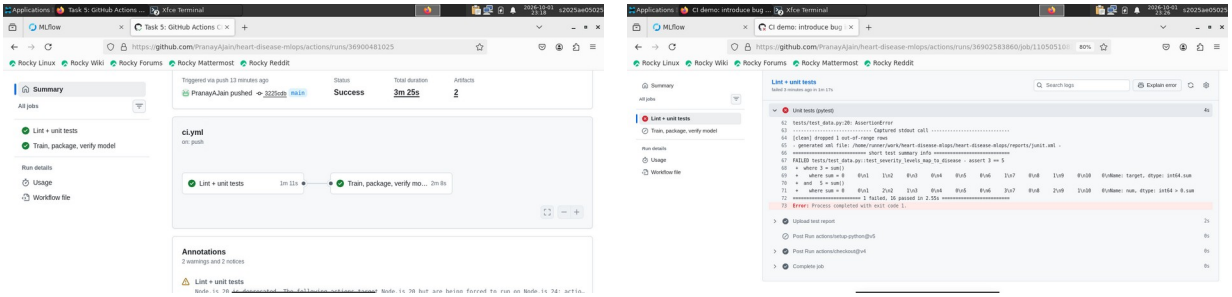


Figure 8 - Green pipeline on main (left); deliberate bug caught by CI (right)

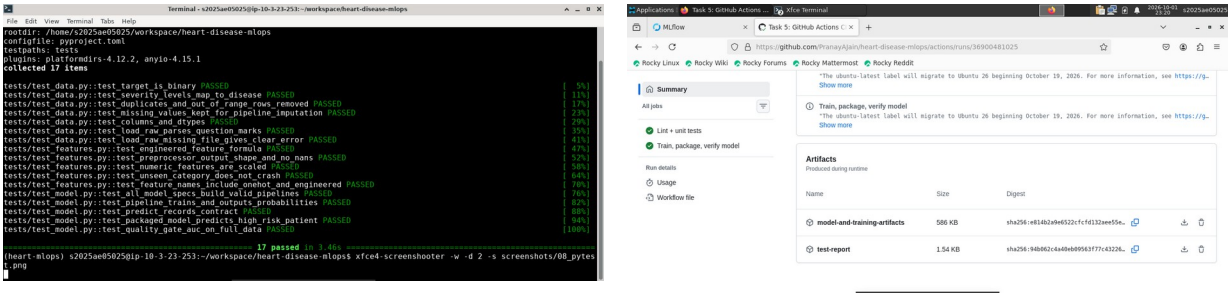


Figure 9 - All unit tests passing locally (left); artifacts uploaded by the CI pipeline (right)

7. Model containerisation

src/heart/api.py is a FastAPI service with /predict (one patient, JSON in, prediction + probability + confidence out), /predict/batch (up to 100 patients), /health, /model-info, /metrics and auto-generated Swagger docs at /docs. Input is validated with Pydantic: types, allowed category codes and physiological

ranges, so invalid input is rejected with HTTP 422 in about 1 ms, before it reaches the model. ca and thal are optional and are imputed by the pipeline.

Dockerfile design choices: python:3.11-slim base; a separate requirements-serve.txt with only runtime packages (no Jupyter, MLflow or XGBoost); dependencies installed before the code is copied so that layer is cached; only the code and the 2 model files are copied (.dockerignore excludes data, notebooks, tests and screenshots); the container runs as a non-root user; and a HEALTHCHECK calls /health.

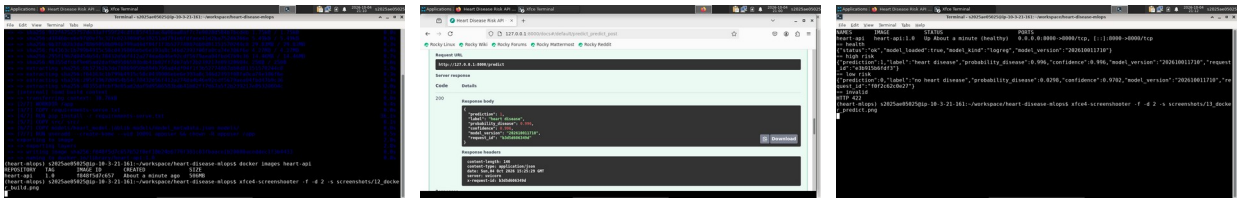


Figure 10 - Docker image built (left); /predict via Swagger UI (middle); container healthy, high-risk 0.996, low-risk 0.970, invalid -> 422 (right)

8. Production deployment on Kubernetes

I deployed to a local minikube cluster (Kubernetes v1.32). The image was loaded into the cluster with minikube image load, so imagePullPolicy is Never and no registry is needed.

- k8s/deployment.yaml: 2 replicas for availability; CPU and memory requests and limits; a readiness probe (traffic only goes to pods whose model is loaded) and a liveness probe (Kubernetes restarts a hung pod), both on /health; Prometheus scrape annotations.
- k8s/service.yaml: a Service that load-balances across the pods (also reachable on NodePort 30080 for direct testing).
- k8s/ingress.yaml: an nginx Ingress (minikube ingress addon) that exposes the API on port 80 of the cluster address, routing all paths to the service. This is the production entry point.

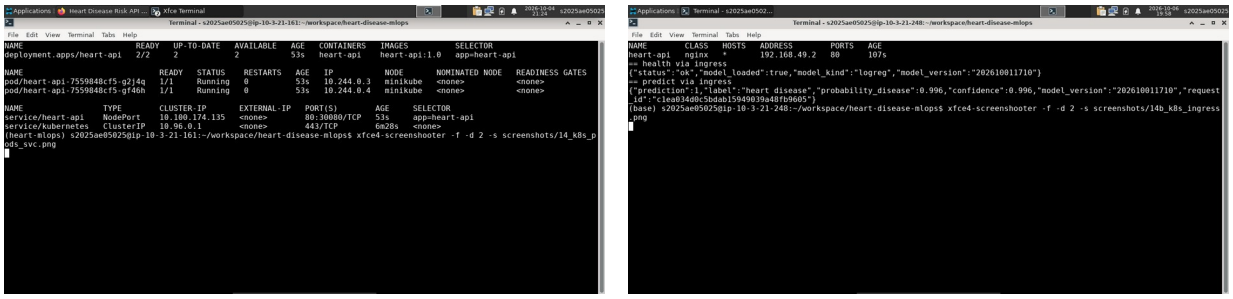


Figure 11 - Deployment 2/2 with two pods and the service (left); API via the nginx Ingress on port 80 (right)

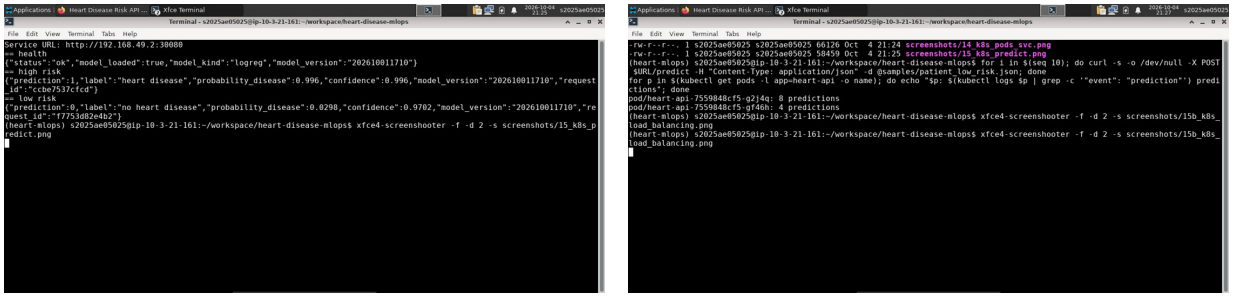


Figure 12 - Endpoint verified through the Kubernetes service (left); requests spread across both pods (right)

A useful side effect: nginx assigns each incoming request an ID and passes it in the X-Request-ID header. The API reuses that header instead of generating its own, so one request can be traced from the Ingress logs through to the API's JSON logs.

9. Monitoring and logging

Logging: every request writes one JSON line to stdout with a request id, method, path, status code, latency and client; every prediction writes a second line with the same request id, the prediction, probability and key patient fields. The request id is also returned in the x-request-id header, so a client complaint can be traced to the exact log lines. JSON lines can be read directly by log tools such as Loki or the ELK stack.

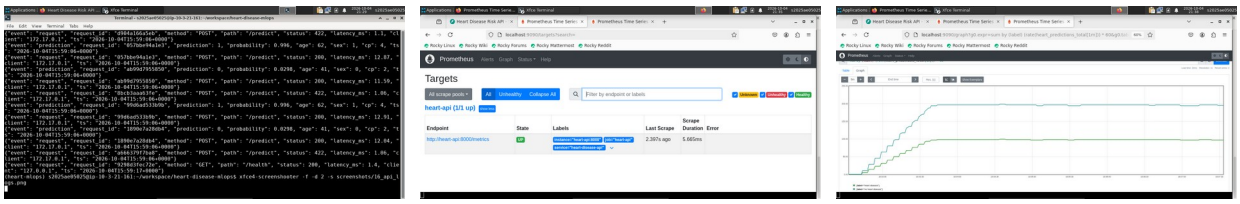


Figure 13 - Structured JSON request logs (left); Prometheus target UP (middle); predictions per minute by outcome in Prometheus (right)

Metrics: the API exposes /metrics with standard HTTP metrics (request count by endpoint and status, latency histogram) plus two model metrics I added: heart_predictions_total by predicted label and heart_prediction_probability, a histogram of predicted probabilities. Prometheus scrapes it every 5 seconds, and Grafana is provisioned automatically (data source and dashboard as code in monitoring/).

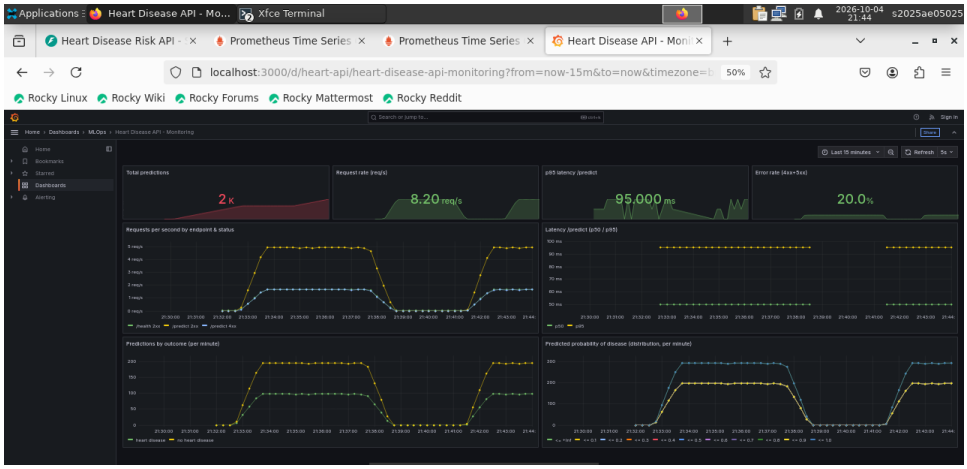


Figure 14 - Grafana dashboard during two traffic bursts

Reading the dashboard: the error rate of about 20% is expected, because the traffic generator deliberately sends one invalid request in every five, which shows that rejected input is visible immediately. Outcomes run at about 2:1 no-disease to disease, matching the low-risk/high-risk mix sent. The probability-distribution panel is the one I would watch for model drift: if real patients' predicted probabilities shift, this panel shows it before accuracy can be measured. One limitation: the p95 latency tile shows about 95 ms although logs show about 12 ms, because Prometheus estimates percentiles inside coarse histogram buckets.

10. Challenges and lessons learned

- **MLflow and SQLAlchemy:** MLflow 2.17 failed against the SQLAlchemy 1.4 already in the environment. I pinned sqlalchemy==2.0.36 in requirements.txt so CI and every rebuild get a working combination.
- **Lab machine reset:** a new lab session came with a fresh host, so the conda environment and git identity were gone while the project folder survived. Rebuilding from requirements.txt + pip install -e . worked first time, a practical check of reproducibility.
- **Local images in Kubernetes:** minikube runs its own container runtime, so the locally built image had to be loaded with minikube image load and the deployment set to imagePullPolicy: Never.
- **Choosing the model on CV, not test:** with 61 test rows, the test ranking differed from the CV ranking; selecting by CV avoids tuning to noise.

11. Repository and how to run

Repository: https://github.com/PranayAJain/heart-disease-mlops **Demo video:** Google Drive video (9.5 min)

Contents: scripts/ (download, verify, traffic), src/heart/ (data, features, train, tracking, package, predict, api), notebooks/ (EDA), tests/ (24 tests), .github/workflows/ci.yml, Dockerfile, k8s/ (deployment, service, ingress), monitoring/ (Prometheus, Grafana), models/, reports/, screenshots/.

Quick start (full instructions in README.md):

```
conda create -n heart-mlops python=3.11 && conda activate heart-mlops
pip install -r requirements.txt && pip install -e .
python scripts/download_data.py && python -m heart.data
python -m heart.train && python -m heart.package && pytest
docker build -t heart-api:1.0 . && docker run -d -p 8000:8000 heart-api:1.0
minikube addons enable ingress && minikube image load heart-api:1.0 && kubectl
apply -f k8s/
curl http://$(minikube ip)/health     # API via the Ingress
python -m heart.predict samples/patient_high_risk.json     # CLI inference
```

API access (local testing)

The API is deployed on a local minikube cluster, so there is no public URL. After the quick start above it is reachable at:

Where	Base URL	Try it
Docker container	http://localhost:8000	curl localhost:8000/health ; Swagger UI at /docs
Kubernetes (nginx Ingress)	http://$(minikube ip) (192.168.49.2)	curl http://$(minikube ip)/health
Predict (either URL)	POST /predict	curl -X POST <url>/predict -H "Content-Type: application/json" -d @samples/patient_high_risk.json